

Programmierung 1

im Wintersemester 2025/26

Praktikumsaufgabe 9

Konstruktoren und Garbage Collection (Kap. 9)

Review: 13. Januar 2026 (DFIW) und 14. Januar 2026 (PI)

In dieser Praktikumsaufgabe arbeiten Sie mit Konstruktoren. Der Schwerpunkt liegt auf der Implementierung von Konstruktoren, deren Überladung und dem Aufruf verschiedener Konstruktortypen in einer Klassenhierarchie.

In **Aufgabe 1** implementieren Sie Konstruktoren für eine Klasse und verwenden diese in einer Klassenhierarchie.

In **Aufgabe 2** erweitern Sie eine Klassenhierarchie um geeignete Konstruktoren, um bereits bei der Erzeugung von Objekten die notwendigen Informationen zu übergeben.

Hinweis: Da das Datum der Klausur immer näher rückt, werden die Aufgaben immer weniger detailliert beschrieben. Auf diese Weise können Sie sich auf die Art der Aufgabenstellungen vorbereiten, die Sie in der Klausur erwarten.

Heads up!

Für diese und die folgenden Praktikumsaufgaben gelten weiterhin die Grundsätze der professionellen Softwareentwicklung, die Sie bereits in den vorherigen Praktikumsaufgaben kennengelernt haben. *Dies wird auch in der Klausur von Ihnen erwartet.* Und nach wie vor gilt: Lösen Sie die Aufgaben nicht allein, sondern mindestens zu zweit! Zum Beispiel mit Ihrer Coding-Partner:in. Denn in der Gruppe lernt es sich besser.

Wir als Ihre Lernbegleiter:innen unterstützen Sie gerne bei der Lösung der Aufgaben.

1 Implementierung und Anwendung von Konstruktoren

Lernziele

- ☐ Sie implementieren Klassen mit Konstruktoren, um Instanzvariablen direkt bei der Objekterzeugung zu initialisieren.
- ☐ Sie erstellen überladene Konstruktoren, um Standardwerte zu setzen.
- ☐ Sie wenden verschiedene Konstruktortypen in einer Klassenhierarchie an.

In dieser Aufgabe implementieren und verwenden Sie verschiedene Konstruktortypen für Klassen in einer Vererbungshierarchie. Sie verwenden diese für die Initialisierung von Objekten, wobei Sie sowohl überladene Konstruktoren als auch Superklassenkonstruktoren verwenden.

1.1 Eine Klasse mit Konstruktor

Implementieren Sie eine Klasse `Car` mit einem Konstruktor, der alle Attribute als Parameter erhält und die Instanzvariablen entsprechend initialisiert!

Die Klasse soll die folgenden Attribute haben:

brand: die Marke als `String`

model: das Modell als `String`

color: die Farbe als `String`

maxSpeed: die Höchstgeschwindigkeit in km/h als `int`

Die Klasse soll eine Methode haben, die Informationen über das Auto auf der Konsole ausgibt. Diese Methode könnte zum Beispiel `printInfo()` heißen.

Schreiben Sie eine Testklasse, die mit Hilfe des Konstruktors ein Auto erzeugt und die Informationen über das Auto auf der Konsole ausgibt.

1.2 Überladene Konstruktoren

Fügen Sie einen Konstruktor hinzu, der die Farbe und die Höchstgeschwindigkeit auf sinnvolle Standardwerte setzt!

Zum Beispiel könnten Autos standardmäßig schwarz sein und eine Höchstgeschwindigkeit von 180 km/h haben.

Hinweis: Verwenden Sie dazu die Überladung von Konstruktoren.

Erweitern Sie die Testklasse, um den neuen Konstruktor zu verwenden und die Informationen über das Auto auf der Konsole anzuzeigen.

1.3 Vererbung und Konstruktoren

Implementieren Sie eine Klasse `ElectricCar`, die von der Klasse `Car` erbt!

Ein Elektroauto hat zusätzlich eine Batteriekapazität (`batteryCapacity`) in kWh (als `int`). Die Standardkapazität beträgt 60 kWh.

Implementieren Sie einen Konstruktor, der alle Attribute als Parameter erhält und diese entsprechend setzt. Implementieren Sie außerdem einen zweiten Konstruktor, der nur die Parameter Marke und Modell entgegennimmt und für Farbe, Geschwindigkeit und Batteriekapazität die Standardwerte setzt.

Schreiben Sie eine Testklasse, die mit Hilfe der beiden Konstruktoren Elektroautos erzeugt und alle Informationen über diese Autos auf der Konsole ausgibt.

1.4 Reflexion im Coding-Team

Diskutieren Sie im Coding-Team, um Ihr Verständnis zu vertiefen!

1.4.1 Konstruktoren und Objektinitialisierung

Welche Vorteile bieten Konstruktoren bei der Initialisierung von Objekten?

- Wie stellen Konstruktoren sicher, dass Objekte in einem gültigen Zustand erzeugt werden?
- Welche Probleme können entstehen, wenn Attribute ohne Konstruktoren initialisiert werden?
- Wie unterstützen Konstruktoren das Prinzip der Datenkapselung?

1.4.2 Überladene Konstruktoren

Wie können überladene Konstruktoren die Flexibilität bei der Erstellung von Objekten erhöhen?

- Wie ermöglichen überladene Konstruktoren verschiedene Arten der Objekterzeugung?
- Welche Vorteile bietet die Verwendung von Standardwerten in überladenen Konstruktoren?
- Wie kann Code-Duplizierung durch geschickten Einsatz von überladenen Konstruktoren vermieden werden?

1.4.3 Konstruktoren in Vererbungshierarchien

Welche Rolle spielen Konstruktoren in einer Vererbungshierarchie und wie werden sie aufgerufen?

- Wie und wann wird der Konstruktor einer Superklasse aufgerufen?
- Welche Bedeutung hat das Schlüsselwort **super** im Konstruktor?

- Wie werden die Attribute einer Superklasse korrekt initialisiert?

2 Ersetzen von Setter-Methoden durch Konstruktoren

Lernziele

- ☐ Sie verwenden Konstruktoren in abstrakten Superklassen, um deren Objekte bei der Erzeugung zu initialisieren.
- ☐ Sie passen Unterklassen einer Superklasse an, um dort den Konstruktor der allgemeinen Superklasse zu verwenden.
- ☐ Sie verwenden Konstruktoren in Unterklassen, um deren spezifische Attribute zu initialisieren.

Strecklernziele

- ☐ Sie verwenden eine Methode, die den Zustand eines Objekts ändert, anstatt eine Setter-Methode zu verwenden.
- ☐ Sie erweitern das Verhalten einer Klasse, indem Sie den aktuellen Zustand des Objekts berücksichtigen.

In dieser Aufgabe erweitern Sie die Klassenhierarchie rund um Tiere und Roboter aus der vorherigen Praktikumsaufgabe um Konstruktoren. Diese sollen die bisher verwendeten Setter-Methoden ersetzen und die Objekte direkt bei der Erzeugung initialisieren.

Dabei können Sie sich an der folgenden Beispielausgabe orientieren:

```
Smaug roars and breathes fire!
Smaug is eating.
Whiskers meows.
Whiskers is eating.
Whiskers licks its paws after eating.
Whiskers is playing with a ball of yarn.
Buddy barks.
Buddy is eating.
Buddy howls after eating.
Buddy is fetching the ball.
Simba roars.
Simba is eating.
Simba licks its paws after eating.
Ghost howls.
Ghost is eating.
Ghost howls after eating.
```

R2-D2 is ready to perform tasks and play.
R2-D2 is guarding the house.
R2-D2 is now discharged.
R2-D2 needs recharging before performing tasks.
R2-D2 cannot play while discharged.
R2-D2 is fully charged.
R2-D2 is playing fetch in simulation mode.
R2-D2 is now discharged.
R2-D2 needs recharging before performing tasks.
R2-D2 is fully charged.
R2-D2 is guarding the house.
R2-D2 is now discharged.
R2-D2 cannot play while discharged.
R2-D2 is fully charged.

Heads up!

Erweitern Sie Ihre Klassenhierarchie in Pseudo-UML um Konstruktoren und neue Methoden, entfernen Sie nicht mehr benötigte Setter-Methoden und passen Sie Ihre Testklasse entsprechend an.

2.1 Superklasse `Animal`

Erstellen Sie in der Klasse `Animal` einen Konstruktor, der alle Attribute anhand der übergebenen Parameter initialisiert!

Entfernen Sie die Setter-Methoden, so dass die Attribute nur noch über den Konstruktor gesetzt werden können, und passen Sie Ihre Testklasse entsprechend an.

2.2 Unterklassen von `Animal`

Passen Sie die Unterklassen von `Animal` an, um den neuen Konstruktor zu verwenden!

Hinweis: Nach dem Hinzufügen des Konstruktors in der Superklasse `Animal` erhalten Sie in den Unterklassen einen Kompilierfehler, da der Standardkonstruktor nicht mehr zur Verfügung

steht. Der Compiler gibt Ihnen einen Hinweis, wie Sie diesen Fehler beheben können: `Must define an explicit constructor`.

Passen Sie auch Ihre Testklasse an, um die neuen Konstruktoren anstelle der Setter-Methoden zu verwenden.

2.3 Superklasse Robot

Erstellen Sie in der Klasse `Robot` einen Konstruktor, der alle Attribute anhand der übergebenen Parameter initialisiert!

Entfernen Sie die Setter-Methoden, so dass die Attribute nur noch über den Konstruktor gesetzt werden können, und passen Sie Ihre Testklasse entsprechend an.

2.4 Unterklasse RoboDog

Implementieren Sie in der Klasse `RoboDog` einen Konstruktor, der alle Attribute anhand der übergebenen Parameter initialisiert, und entfernen Sie alle Setter-Methoden!

Nach der Instanziierung eines `RoboDog`-Objekts soll sich der Roboter in einem geladenen Zustand befinden.

Optional: Ersetzen Sie die Methode `setCharged` durch die Methode `void charge()`, damit sich der Roboter wieder aufladen lässt und kein “Wegwerfprodukt” ist.¹ Die Methoden `performTask` und `play` sollen anschließend die jeweilige Aktion im aufgeladenen Zustand ausführen und danach den Batteriestatus auf “entladen” setzen, so dass der Roboter keine weiteren Aktionen ausführen kann, bis er wieder aufgeladen ist.

Passen Sie Ihre Testklasse an, um die neuen Konstruktoren und Methoden zu verwenden.

¹Nachdem Sie `setCharged` entfernt haben, können Sie den Ladezustand nicht mehr direkt verändern und den Roboter somit auch nicht mehr aufladen.

2.5 Reflexion im Coding-Team

Diskutieren Sie im Coding-Team, um Ihr Verständnis zu vertiefen!

2.5.1 Unterschiede zwischen Konstruktoren und Setter-Methoden

Warum sind Konstruktoren hier besser geeignet als Setter-Methoden?

- Was ist der grundsätzliche Unterschied zwischen Konstruktoren und Setter-Methoden?
- Welche Vorteile hat die Initialisierung durch Konstruktoren in diesem Beispiel?
- Wie wird die Objekterzeugung durch die Verwendung von Konstruktoren sicherer?

2.5.2 Konstruktoren in der Vererbungshierarchie

Wie funktioniert das Zusammenspiel der Konstruktoren in der Vererbungshierarchie?

- Warum muss der Konstruktor der Superklasse in den Unterklassen aufgerufen werden?
- Wie wird sichergestellt, dass die Attribute der Superklasse korrekt initialisiert werden?
- Welche Rolle spielt die Reihenfolge der Initialisierung in der Vererbungshierarchie?

2.5.3 Zustandsänderung durch Methoden statt durch Setter

Wie verändert die Zustandsänderung durch Methoden das Objektverhalten im Vergleich zu Setter-Methoden?

- Was ist der Unterschied zwischen der Modellierung des Batteriezustands mit der Methode `charge` und der Verwendung eines Setters?

- Welche Vorteile bietet die automatische Zustandsänderung nach Ausführung einer Aktion?
- Wie wird das Verhalten des RoboDog durch diesen Ansatz realistischer modelliert?